

**LAPORAN PRAKTIKUM  
BASIS DATA NOSQL**

**Aggregation Framework**

**Disusun oleh:**

**Nama : Renisa Rahmawati**

**NIM : 225443025**

**Kelas : 1 AEC-1**

**Program Studi : Teknologi Rekayasa Informatika Industri**

**Jurusan : Teknik Otomasi Manufaktur dan Mekatronika**

**Institusi : Politeknik Manufaktur Bandung**

**Tahun Ajaran 2025/2026**

## **BAB I**

### **PENDAHULUAN**

Praktikum ini memiliki tujuan utama untuk meningkatkan pemahaman mengenai mekanisme kerja Aggregation Framework pada MongoDB sebagai metode pengolahan data NoSQL. Praktikum ini bertujuan agar praktikan mampu mengimplementasikan berbagai tahapan pipeline agregasi, seperti \$match untuk penyaringan data, \$group untuk pengolahan statistik, hingga \$lookup dan \$unwind untuk penggabungan antar koleksi. Selain penguasaan teknis, tujuan lainnya adalah melatih kemampuan analisis dalam memahami perintah Aggregation. Dengan tercapainya tujuan tersebut, praktikan diharapkan dapat menggunakan hasil pengolahan data sebagai landasan untuk melakukan deteksi, pemeliharaan preventif, serta pengambilan keputusan strategis dalam menjaga stabilitas operasional di lingkungan industri.

## BAB II

### DESAIN DATA TUTORIAL TERBIMBING

#### 2.1. Membuat Database

Kode:

```
use studi kasus_oe
```

Output:

```
< switched to db studi_kasus_oe  
studi_kasus_oe >
```

#### 2.2. Membuat Dokumen

Kode:

```
var docs = [];  
var mesinList = []; for (let i=1;i<=10;i++)  
mesinList.push("M"+i.toString().padStart(2,'0'));  
var start = new Date(2026,3,1); // April  
for (let i=0; i<1000; i++) {  
  var tgl = new Date(start.getTime() + Math.floor(Math.random()*90)*86400000);  
  var shift = Math.floor(Math.random()*3)+1;  
  var target = Math.floor(Math.random()*301)+200;  
  var actual_ok = Math.floor(target * (0.6 + Math.random()*0.4));  
  var actual_reject = Math.floor(actual_ok * Math.random()*0.15);  
  var durasi_tersedia = 480;  
  var durasi_operasi = Math.floor(durasi_tersedia * (0.7 + Math.random()*0.3));  
  docs.push({  
    mesin: mesinList[Math.floor(Math.random()*10)],  
    tanggal: tgl,  
    shift: shift,  
    target: target,  
    actual_ok: actual_ok,  
    actual_reject: actual_reject,  
    durasi_operasi_menit: durasi_operasi,  
    durasi_tersedia_menit: durasi_tersedia  
  });  
  if (docs.length === 500) { db.produksi_harian.insertMany(docs); docs = []; }  
}  
if (docs.length) db.produksi_harian.insertMany(docs);
```

Output:

```
< []  
studi_kasus_oe >
```

Syntax di atas menghasilkan data dummy produksi harian sebanyak 1000 dokumen di koleksi produksi\_harian dengan kategori yang telah ditentukan.

### 2.3. Memverifikasi Dokumen

Kode:

```
db.produksi_harian.countDocuments()
```

Output:

```
> db.produksi_harian.countDocuments()  
< 1000
```

Kode:

```
db.produksi_harian.findOne()
```

Output:

```
< {  
  _id: ObjectId('69f80fdacea939b6534fa467'),  
  mesin: 'M04',  
  tanggal: 2026-04-25T17:00:00.000Z,  
  shift: 3,  
  target: 316,  
  actual_ok: 223,  
  actual_reject: 25,  
  durasi_operasi_menit: 353,  
  durasi_tersedia_menit: 480  
}
```

## BAB III

### PIPELINE DAN HASIL TUTORIAL TERBIMBING

#### 3.1. Pipeline Lengkap

Syntax:

```
db.produksi_harian.aggregate([
  // Tahap 1: Tidak perlu $match karena ingin semua data
  // Tahap 2: Group by mesin dan bulan
  { $group: {
    _id: {
      mesin: "$mesin",
      bulan: { $month: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  // Tahap 3: Hitung OEE
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    total_ok: 1,
    total_target: 1,
    total_reject: 1,
    availability: { $divide: ["$total_op", "$total_avail"] },
    performance: { $divide: ["$total_ok", "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  // Tahap 4: Urutkan berdasarkan OEE terendah
  { $sort: { OEE: 1 } }
])
```

Output:

```
< (  
  total_target: 482,  
  total_ok: 272,  
  total_reject: 39,  
  mesin: 'M18',  
  bulan: 3,  
  tahun: 2026,  
  availability: 0.74375,  
  performance: 0.6766169154228856,  
  quality: 0.8745988787395499,  
  OEE: 0.4481273375885845  
)
```

```
(  
  total_target: 836,  
  total_ok: 591,  
  total_reject: 19,  
  mesin: 'M85',  
  bulan: 3,  
  tahun: 2026,  
  availability: 0.8427883333333333,  
  performance: 0.7869377998438622,  
  quality: 0.9688524598163935,  
  OEE: 0.5771864643795591  
)
```

```
(  
  total_target: 7878,  
  total_ok: 5836,  
  total_reject: 379,  
  mesin: 'M85',  
  bulan: 6,  
  tahun: 2026,  
  availability: 0.8446822727272727,  
  performance: 0.7487971566387488,  
  quality: 0.9398185836282735,  
  OEE: 0.5875241227631643  
)
```

```
(
```

```
(  
  total_target: 251,  
  total_ok: 171,  
  total_reject: 3,  
  mesin: 'M81',  
  bulan: 3,  
  tahun: 2026,  
  availability: 0.8833333333333333,  
  performance: 0.6812749883984863,  
  quality: 0.9827586286896551,  
  OEE: 0.591417898259651  
)
```

```
(  
  total_target: 732,  
  total_ok: 588,  
  total_reject: 74,  
  mesin: 'M88',  
  bulan: 3,  
  tahun: 2026,  
  availability: 0.8416666666666667,  
  performance: 0.7923497267759563,  
  quality: 0.8868501529851988,  
  OEE: 0.5914353592575882  
)
```

```
(  
  total_target: 18867,  
  total_ok: 7713,  
  total_reject: 550,  
  mesin: 'M89',  
  bulan: 6,  
  tahun: 2026,  
  availability: 0.8447538864197531,  
  performance: 0.7661666832224099,  
  quality: 0.9334382185646859,  
  OEE: 0.6841414438811488  
)
```

```
)  
(  
  total_target: 15224,  
  total_ok: 11632,  
  total_reject: 853,  
  mesin: 'M18',  
  bulan: 4,  
  tahun: 2026,  
  availability: 0.8534883728938233,  
  performance: 0.7648567524968589,  
  quality: 0.9316788136163396,  
  OEE: 0.6875598685261263  
)
```

```
(  
  total_target: 15823,  
  total_ok: 11761,  
  total_reject: 931,  
  mesin: 'M83',  
  bulan: 4,  
  tahun: 2026,  
  availability: 0.8453252832528325,  
  performance: 0.7828662717167811,  
  quality: 0.9266467865868264,  
  OEE: 0.6132338978589186  
)
```

```
(  
  total_target: 18516,  
  total_ok: 8151,  
  total_reject: 677,  
  mesin: 'M85',  
  bulan: 5,  
  tahun: 2026,  
  availability: 0.8681293183448275,  
  performance: 0.7751846825184682,  
  quality: 0.9233121884911645,  
  OEE: 0.615563175791454  
)
```

```
(  
  total_target: 13576,  
  total_ok: 18684,  
  total_reject: 828,  
  mesin: 'M18',  
  bulan: 6,  
  tahun: 2026,  
  availability: 0.8435416666666666,  
  performance: 0.786977818267531,  
  quality: 0.9287284458625869,  
  OEE: 0.6165291223591893  
)
```

Berikut merupakan penjelasan mengenai stage yang dipakai:

1. Stage \$group  
Stage ini mengelompokkan data berdasarkan mesin, bulan dan tahun. Perintah \$sum menjumlahkan seluruh angka field target, actual\_ok, actual\_reject, serta durasi waktu untuk mendapatkan total performa.
2. Stage \$project  
Stage ini membentuk ulang dokumen.
3. Stage \$sort  
Stage ini mengurutkan hasil akhir berdasarkan nilai OEE dari yang terkecil hingga terbesar.

### 3.2. Menemukan Mesin Dengan Oee Di Bawah 80%

Menambahkan \$match langsung setelah \$project pada pipeline di atas.

Syntax:

```
db.produksi_harian.aggregate([
// Tahap 1: Tidak perlu $match karena ingin semua data
// Tahap 2: Group by mesin dan bulan
{ $group: {
  _id: {
    mesin: "$mesin",
    bulan: { $month: "$tanggal" },
    tahun: { $year: "$tanggal" }
  },
  total_target: { $sum: "$target" },
  total_ok: { $sum: "$actual_ok" },
  total_reject: { $sum: "$actual_reject" },
  total_op: { $sum: "$durasi_operasi_menit" },
  total_avail: { $sum: "$durasi_tersedia_menit" }
}},
// Tahap 3: Hitung OEE
{ $project: {
  _id: 0,
  mesin: "$_id.mesin",
  bulan: "$_id.bulan",
  tahun: "$_id.tahun",
  total_ok: 1,
  total_target: 1,
  total_reject: 1,
  availability: { $divide: ["$total_op", "$total_avail"] },
  performance: { $divide: ["$total_ok", "$total_target"] },
  quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
  OEE: {
    $multiply: [
      { $divide: ["$total_ok", "$total_target"] },
```

```

    { $divide: ["$total_op", "$total_avail"] },
    { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
  ]
}
}},
// Menemukan mesin dengan OEE di bawah 80%
{ $match: { OEE: { $lt: 0.80 } } },
// Tahap 4: Urutkan berdasarkan OEE terendah
{ $sort: { OEE: 1 } }
])

```

Output:

```

< {
  total_target: 482,
  total_ok: 272,
  total_reject: 39,
  mesin: 'M10',
  bulan: 3,
  tahun: 2026,
  availability: 0.74375,
  performance: 0.6766169154228856,
  quality: 0.8745980707395499,
  OEE: 0.4401273375885845
}
{
  total_target: 836,
  total_ok: 591,
  total_reject: 19,
  mesin: 'M05',
  bulan: 3,
  tahun: 2026,
  availability: 0.8427083333333333,
  performance: 0.7069377990430622,
  quality: 0.9688524590163935,
  OEE: 0.5771864643795591
}
{
  total_target: 7878,
  total_ok: 5836,
  total_reject: 379,
  mesin: 'M05',
  bulan: 6,
  tahun: 2026,
  availability: 0.8446022727272727,
  performance: 0.7407971566387488,
  quality: 0.9390185036202735,
  OEE: 0.5875241227631643
}

```

```

{
  total_target: 251,
  total_ok: 171,
  total_reject: 3,
  mesin: 'M01',
  bulan: 3,
  tahun: 2026,
  availability: 0.8833333333333333,
  performance: 0.6812749003984063,
  quality: 0.9827586206896551,
  OEE: 0.591417090259651
}
{
  total_target: 732,
  total_ok: 580,
  total_reject: 74,
  mesin: 'M08',
  bulan: 3,
  tahun: 2026,
  availability: 0.8416666666666667,
  performance: 0.7923497267759563,
  quality: 0.8868501529051988,
  OEE: 0.5914353592575882
}
{
  total_target: 10067,
  total_ok: 7713,
  total_reject: 550,
  mesin: 'M09',
  bulan: 6,
  tahun: 2026,
  availability: 0.8447530864197531,
  performance: 0.7661666832224099,
  quality: 0.9334382185646859,
  OEE: 0.6041414430011408
}

```



```

{
  total_target: 15224,
  total_ok: 11632,
  total_reject: 853,
  mesin: 'M10',
  bulan: 4,
  tahun: 2026,
  availability: 0.8534883720930233,
  performance: 0.7640567524960589,
  quality: 0.9316780136163396,
  OEE: 0.6075598605261263
}
{
  total_target: 15023,
  total_ok: 11761,
  total_reject: 931,
  mesin: 'M03',
  bulan: 4,
  tahun: 2026,
  availability: 0.8453252032520325,
  performance: 0.7828662717167011,
  quality: 0.9266467065868264,
  OEE: 0.6132330978589106
}
}

{
  total_target: 10516,
  total_ok: 8151,
  total_reject: 677,
  mesin: 'M05',
  bulan: 5,
  tahun: 2026,
  availability: 0.8601293103448275,
  performance: 0.7751046025104602,
  quality: 0.9233121884911645,
  OEE: 0.615563175791454
}
{
  total_target: 13576,
  total_ok: 10684,
  total_reject: 820,
  mesin: 'M10',
  bulan: 6,
  tahun: 2026,
  availability: 0.8435416666666666,
  performance: 0.786977018267531,
  quality: 0.9287204450625869,
  OEE: 0.6165291223591893
}
}

```

Stage \$match memfilter dokumen dan meloloskan dokumen yang nilai OEE-nya kurang dari 0.80 (80%).

### 3.3. Distribusi Jumlah Produksi Per Shift Dalam Bucket

#### 1. Sebaran jumlah produksi

Manajemen ingin melihat sebaran jumlah produksi (actual\_ok) per shift, misal dikelompokkan menjadi bucket: rendah (0-200), sedang (200-400), tinggi (400+). Gunakan \$bucket.

Syntax:

```

db.produksi_harian.aggregate([
  { $bucket: {
    groupBy: "$actual_ok",
    boundaries: [0, 200, 300, 400, 600], // 0-199, 200-299, 300-399, 400-600
    default: "di atas 600",
    output: {
      count: { $sum: 1 },
      rata_reject: { $avg: "$actual_reject" }
    }
  }
}]
)

```

Output:

```
< {
  _id: 0,
  count: 196,
  rata_reject: 12.311224489795919
}
{
  _id: 200,
  count: 390,
  rata_reject: 18.51794871794872
}
{
  _id: 300,
  count: 324,
  rata_reject: 24.212962962962962
}
{
  _id: 400,
  count: 90,
  rata_reject: 34.28888888888889
}
```

Stage \$bucket melakukan pengelompokkan sekaligus kategorisasi data berdasarkan nilai tertentu dengan kategori "\$actual\_ok" lalu menghitung berapa kali kejadian produksi dan berapa rata-rata produk reject dalam rentang produksi tersebut.

## 2. Distribusi per shift

Untuk distribusi per shift, kita bisa \$group atau \$facet. Misal kita ingin tiga bucket terpisah untuk setiap shift:

Syntax:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: "$shift",
    data: { $push: "$actual_ok" } // kumpulkan nilai
  }},
  // Kemudian di aplikasi atau gunakan operator $bucket di sub-pipeline (tidak bisa
  langsung karena $bucket butuh array dokumen)
])
```

Output:

```
_id: 1,
data: [
  262,
  290,
  309,
  332,
  374,
  440,
  192,
  231,
  281,
  197,
  302,
  280,
  126,
  220,
  228,
  487,
  422,
  384,
  352,
  275,
  304,
  281,
  412,
  277,
  295,
  364,
  140,
  357,
  331,
  166,
  404,
  160,
  266,
  271,
  341,
  407,
  256,
  309,
  188,
  423,
  ... 230 more items
]
}
{
  _id: 3,
  data: [
    223,
    274,
    354,
    419,
    401,
    226,
    210,
    483,
    230,
    281,
    199,
    315,
    214,
    398,
    329,
    184,
    179,
    333,
```

```
324,
446,
203,
... 233 more items
]
}
{
  _id: 2,
  data: [
    349,
    211,
    287,
    194,
    314,
    202,
    328,
    203,
    250,
    331,
    281,
    254,
    244,
    187,
    307,
    283,
    296,
    423,
    249,
    244,
    294,
    201,
    293,
    234,
    326,
    268,
    446,
    253,
    289,
    177,
    365,
    327,
    229,
    153,
    360,
    174,
    337,
    218,
    227,
    210,
    ... 237 more items
  ]
}
```

Stage \$group memecah data berdasarkan nilai field shift (misalnya Shift 1, 2, dan 3). Hasil akhirnya adalah satu dokumen per shift.

Alternatif: Gunakan \$facet untuk menjalankan pipeline bucket paralel per shift:

Syntax:

```
db.produksi_harian.aggregate([
  { $facet: {
    "shift1": [
      { $match: { shift: 1 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+",
output: { count: {$sum:1} } } } }
    ],
    "shift2": [
      { $match: { shift: 2 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+",
output: { count: {$sum:1} } } } }
    ],
    "shift3": [
      { $match: { shift: 3 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+",
output: { count: {$sum:1} } } } }
    ]
  } }
])
```

Output:

```
< {
  shift1: [
    {
      _id: 0,
      count: 65
    },
    {
      _id: 200,
      count: 126
    },
    {
      _id: 300,
      count: 104
    },
    {
      _id: 400,
      count: 35
    }
  ],
  shift2: [
    {
      _id: 0,
      count: 63
    },
    {
      _id: 200,
      count: 136
    },
  ],
  shift3: [
    {
      _id: 300,
      count: 115
    },
    {
      _id: 400,
      count: 23
    }
  ],
  shift3: [
    {
      _id: 0,
      count: 68
    },
    {
      _id: 200,
      count: 128
    },
    {
      _id: 300,
      count: 105
    },
    {
      _id: 400,
      count: 32
    }
  ]
}
```

\$facet berfungsi untuk menjalankan beberapa analisis berbeda secara paralel terhadap satu set data yang sama. Pencarian dibagi menjadi tiga sub-pipeline yaitu shift1, shift2, dan shift3. Setiap sub pipeline melakukan penyaringan data sesuai nomor shift, hasil akhirnya merupakan dokumen yang berisi tiga array terpisah.

### 3. Cara lain – lebih sederhana

Syntax:

```
db.produksi_harian.aggregate([
  { $group: { _id: "$shift", total_produksi: { $sum: "$actual_ok" } } }
])
```

Output:

```
< {
  _id: 1,
  total_produksi: 93047
}
{
  _id: 3,
  total_produksi: 92900
}
{
  _id: 2,
  total_produksi: 94339
}
```

Stage ini berfungsi untuk meringkas data produksi berdasarkan kategori tertentu yaitu \$sum: "\$actual\_ok".

### 3.4. Optimasi dan Analisis

Syntax:

```
db.produksi_harian.createIndex({ mesin: 1, tanggal: 1 })
```

Output:

```
> db.produksi_harian.createIndex({ mesin: 1, tanggal: 1 })
< mesin_1_tanggal_1
```

Syntax di atas menyimpan kombinasi field mesin dan tanggal dalam struktur yang terurut.

### 3.5. Latihan Tambahan

1. Tren bulanan: Modifikasi pipeline agar menampilkan OEE per bulan untuk semua mesin, lalu urutkan bulan.

Syntax:

```
db.produksi_harian.aggregate([
  {
    $group: {
      _id: { $dateToString: { format: "%Y-%m", date: "$tanggal" } },
      total_operasi: { $sum: "$durasi_operasi_menit" },
      total_tersedia: { $sum: "$durasi_tersedia_menit" },
      total_ok: { $sum: "$actual_ok" },
      total_reject: { $sum: "$actual_reject" },
      total_target: { $sum: "$target" }
    }
  },
  {
    $project: {
      availability: { $divide: ["$total_operasi", "$total_tersedia"] },
      performance: { $divide: [{ $add: ["$total_ok", "$total_reject"] },
        "$total_target"] },
      quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
    }
  },
  {
    oee: {
      $multiply: [
        { $divide: ["$total_operasi", "$total_tersedia"] },
        { $divide: [{ $add: ["$total_ok", "$total_reject"] }, "$total_target"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }
],
{
  $sort: {
    _id: 1
  }
})
```

```
{
  $sort: { _id: 1 }
}
```

Output:

```
< {
  _id: '2026-03',
  availability: 0.8242424242424242,
  performance: 0.8786650774731823,
  quality: 0.9226804123711341,
  oee: 0.6682356340520822
}
{
  _id: '2026-04',
  availability: 0.8543413561076605,
  performance: 0.8570281691383402,
  quality: 0.9321529672139585,
  oee: 0.6825173766528136
}
{
  _id: '2026-05',
  availability: 0.8574737344794652,
  performance: 0.8653125807284939,
  quality: 0.928732693958279,
  oee: 0.6891036940849933
}
{
  _id: '2026-06',
  availability: 0.8471108490566037,
  performance: 0.844896473265074,
  quality: 0.934818437409111,
  oee: 0.6690691577049391
}
```

Berikut merupakan penjelasan mengenai stage yang dipakai:

- Stage \$group  
Stage ini mengubah format tanggal menjadi string bulan, lalu semua angka dari seluruh mesin yang beroperasi di bulan tersebut dijumlahkan.
- Stage \$project  
Stage ini memproses data dari angka mentah menjadi perhitungan efisiensi industri yang meliputi availability, performance, dan quality.
- Stage \$sort  
Stage ini mengurutkan hasil akhir berdasarkan field bulan dari bulan terlama hingga bulan terbaru.

2. Mesin terburuk: Tampilkan 3 mesin dengan OEE terendah di bulan Juni 2026 saja (gunakan \$match di awal untuk filter bulan).

Syntax:

```
db.produksi_harian.aggregate([
  {
    $match: {
      tanggal: {
        $gte: ISODate("2026-06-01T00:00:00Z"),
        $lt: ISODate("2026-07-01T00:00:00Z")
      }
    },
    {
      $group: {
        _id: "$mesin",
        total_operasi: { $sum: "$durasi_operasi_menit" },
        total_tersedia: { $sum: "$durasi_tersedia_menit" },
        total_ok: { $sum: "$actual_ok" },
        total_reject: { $sum: "$actual_reject" },
        total_target: { $sum: "$target" }
      }
    },
    {
      $project: {
        oee: {
          $multiply: [
            { $divide: ["$total_operasi", "$total_tersedia"] },
            { $divide: [ { $add: ["$total_ok", "$total_reject"] }, "$total_target" ] },
            { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
          ]
        }
      }
    },
    {
      $sort: { oee: 1 }
    },
    {
      $limit: 3
    }
  ]
})
```



Output:

```
< {
  _id: 'M05',
  oee: 0.6256789621269819
}
{
  _id: 'M09',
  oee: 0.6472216703641159
}
{
  _id: 'M06',
  oee: 0.6599553826094042
}
```

Stage \$match melakukan filter data agar hanya mengambil transaksi produksi yang terjadi di 1 Juni hingga sebelum 1 Juli 2026. Stage \$group mengakumulasi total waktu produksi, waktu tersedia, jumlah produksi (OK), produk gagal, dan target produksi untuk setiap unit, sedangkan \$project menghitung skor akhir OEE dengan mengalikan rasio availability, performance dan quality setiap mesin. Stage \$sort dan \$limit mengurutkan daftar mesin dari nilai OEE terbesar ke terkecil, kemudian sistem hanya mengambil tiga mesin teratas.

3. Kinerja harian: Hitung OEE per mesin per hari (gunakan \$dateToString untuk ekstrak tanggal).

Syntax:

```
db.produksi_harian.aggregate([
  {
    $group: {
      _id: {
        mesin: "$mesin",
        tanggal: { $dateToString: { format: "%Y-%m-%d", date: "$tanggal" } }
      }
    },
    total_operasi: { $sum: "$durasi_operasi_menit" },
    total_tersedia: { $sum: "$durasi_tersedia_menit" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_target: { $sum: "$target" }
  },
  {
    $project: {
      oee: {
        $multiply: [
          { $divide: ["$total_operasi", "$total_tersedia"] },

```

```

    { $divide: [{ $add: ["$total_ok", "$total_reject"] }, "$total_target"] },
    { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] }] }
  ]
}
},
{
  $sort: { "_id.tanggal": 1, "_id.mesin": 1 }
}
])

```

Output:

```

< (
  _id: {
    mesin: 'M01',
    tanggal: '2026-03-31'
  },
  oee: 0.601792828685259
)
(
  _id: {
    mesin: 'M03',
    tanggal: '2026-03-31'
  },
  oee: 0.6716814159292835
)
(
  _id: {
    mesin: 'M04',
    tanggal: '2026-03-31'
  },
  oee: 0.7527547393364928
)
(
  _id: {
    mesin: 'M05',
    tanggal: '2026-03-31'
  },
  oee: 0.5957423744019138
)
(
  _id: {
    mesin: 'M06',
    tanggal: '2026-03-31'
  },
  oee: 0.7548996425379883
)
(
  _id: {
    mesin: 'M08',
    tanggal: '2026-03-31'
  },
  oee: 0.6668943533697632
)
(
  _id: {
    mesin: 'M09',
    tanggal: '2026-03-31'
  },
  oee: 0.8262272727272727
)
(
  _id: {
    mesin: 'M10',
    tanggal: '2026-03-31'
  },
  oee: 0.5832338388457712
)
(
  _id: {
    mesin: 'M02',
    tanggal: '2026-04-01'
  },
  oee: 0.5863675458715596
)
(
  _id: {
    mesin: 'M03',
    tanggal: '2026-04-01'
  },
  oee: 0.6557365953512941
)
)

```

Stage \$group yang melakukan pengelompokan data berdasarkan dua kriteria, yaitu identitas mesin dan tanggal. Di dalam stage ini, dilakukan banyak akumulasi seperti { \$sum: "\$durasi\_operasi\_menit" } menjadi total\_operasi dan seterusnya. Pada stage \$project, sistem melakukan kalkulasi OEE. Tahap \$sort berfungsi untuk menampilkan data terurut berdasarkan tanggal dan identitas mesin.

4. Simulasi perbaikan: Jika quality harus minimal 95%, tampilkan mesin yang tidak memenuhi kriteria tersebut.

Syntax:

```
db.produksi_harian.aggregate([
  {
    $group: {
      _id: "$mesin",
      total_ok: { $sum: "$actual_ok" },
      total_reject: { $sum: "$actual_reject" }
    }
  },
  {
    $project: {
      quality: {
        $divide: [
          "$total_ok",
          { $add: ["$total_ok", "$total_reject"] }
        ]
      }
    }
  },
  {
    $match: {
      quality: { $lt: 0.95 }
    }
  },
  {
    $sort: { quality: 1 }
  }
])
```

Output:

```
< {
  _id: 'M10',
  quality: 0.9288373785728283
}
{
  _id: 'M04',
  quality: 0.9290445021108485
}
{
  _id: 'M03',
  quality: 0.929209273500662
}
{
  _id: 'M08',
  quality: 0.931138005441411
}
{
  _id: 'M07',
  quality: 0.9314946922893129
}
```

```
{
  _id: 'M06',
  quality: 0.9321903590374414
}
{
  _id: 'M05',
  quality: 0.9326348437854604
}
{
  _id: 'M09',
  quality: 0.9332119399102555
}
{
  _id: 'M02',
  quality: 0.9340365682137834
}
{
  _id: 'M01',
  quality: 0.9354222639432881
}
```

Stage \$group mengumpulkan seluruh data produksi berdasarkan identitas mesin lalu menjumlahkan total produk actual\_ok dan actual\_reject. Pada stage \$project, sistem menghitung rasio quality, kemudian \$match memfilter mesin-mesin yang memiliki skor kualitas di bawah 0.95 atau 95%. Stage \$sort mengurutkan daftar mesin berdasarkan persentase kualitas yang paling rendah.

## **BAB IV**

### **ANALISIS TUTORIAL TERBIMBING**

Database studi\_kasus\_oeo berfungsi sebagai instrumen pemantauan kinerja harian sepuluh mesin produksi. Temuan data menunjukkan adanya variasi pada indikator availability, performance, dan quality yang dikalkulasi menjadi nilai OEE untuk setiap shift. Terindikasikan bahwa mesin dengan nilai OEE rendah memerlukan perhatian khusus, terutama jika kegagalan disebabkan oleh tingginya angka actual\_reject, sehingga pemantauan ulang pada komponen untuk meminimalisir pemborosan material.

## BAB V

### DESAIN DATA TUGAS MANDIRI

#### 5.1. Membuat Database

Syntax:

```
use tugas5 225443025
```

Output:

```
> use tugas5_225443025
< switched to db tugas5_225443025
```

#### 5.2. Membuat Dokumen

Syntax:

```
var NIM = "225443025";
var dbName = "tugas5_" + 225443025;
db = db.getSiblingDB(dbName);

db.inspeksi.drop();
db.target_kualitas.drop();

var mesinList = [];
for (let i = 1; i <= 10; i++) mesinList.push("M" + i.toString().padStart(2, '0'));

var batchIds = [];
for (let i = 1; i <= 2000; i++) batchIds.push("B" + i.toString().padStart(4, '0'));

var inspektorList = ["Andi", "Budi", "Citra", "Dian", "Eka"];
var jenisCacatOptions = ["gores", "retak", "bengkok", "warna", "dimensi", "pori"];

var docsInspeksi = [];
var startDate = new Date(2026, 3, 1, 6, 0); // April 2026

for (let i = 0; i < 1500; i++) {
  // Simulasi randomisasi waktu (90 hari)
  var randomMinutes = Math.floor(Math.random() * (60 * 24 * 90));
  var tanggal = new Date(startDate.getTime() + randomMinutes * 60000);

  // Tentukan shift berdasarkan jam
  var jam = tanggal.getHours();
  var shift = (jam < 14) ? 1 : (jam < 22 ? 2 : 3);

  var jumlah = Math.floor(Math.random() * 401) + 100; // 100-500
  var reject = Math.floor(jumlah * (Math.random() * 0.10)); // 0-10% reject

  // Ambil 1-3 jenis cacat secara acak
```

```

var shuffledCacat = jenisCacatOptions.sort(() => 0.5 - Math.random());
var jenis = shuffledCacat.slice(0, Math.floor(Math.random() * 3) + 1);

docsInspeksi.push({
  batch_id: batchIds[Math.floor(Math.random() * batchIds.length)],
  mesin: mesinList[Math.floor(Math.random() * mesinList.length)],
  tanggal: tanggal,
  shift: shift,
  inspektor: inspektorList[Math.floor(Math.random() * inspektorList.length)],
  jumlah_diperiksa: jumlah,
  cacat_ditemukan: reject,
  jenis_cacat: jenis
});

if (docsInspeksi.length === 500) {
  db.inspeksi.insertMany(docsInspeksi);
  docsInspeksi = [];
}
}
if (docsInspeksi.length > 0) db.inspeksi.insertMany(docsInspeksi);
print("Data inspeksi selesai. Total: " + db.inspeksi.countDocuments({}));

var docsTarget = [];
for (let i = 1; i <= 2000; i++) {
  var batch = "B" + i.toString().padStart(4, '0');
  var targetCacat = Math.floor(Math.random() * 15) + 1; // 1-15

  docsTarget.push({
    batch_id: batch,
    target_maks_cacat: targetCacat
  });

  if (docsTarget.length === 500) {
    db.target_kualitas.insertMany(docsTarget);
    docsTarget = [];
  }
}
if (docsTarget.length > 0) db.target_kualitas.insertMany(docsTarget);
print("Data target kualitas selesai. Total: " + db.target_kualitas.countDocuments({}));

```

Output:

```

< Data inspeksi selesai. Total: 1500
< Data target kualitas selesai. Total: 2000

```

### 5.3. Memastikan Data Siap

#### 1. Database

Syntax:

```
use tugas5_225443025
```

Output:

```
> use tugas5_225443025
< switched to db tugas5_225443025
```

#### 2. Inspeksi

Syntax:

```
db.inspeksi.countDocuments()
```

Output:

```
> db.inspeksi.countDocuments()
< 1500
```

#### 3. Jumlah data target kualitas

Kode:

```
db.target_kualitas.countDocuments()
```

Output:

```
> db.target_kualitas.countDocuments()
< 2000
```

#### 4. Struktur dokumen

Syntax:

```
db.inspeksi.findOne()
```

Output:

```
< {
  _id: ObjectId('69f84fe45c9f8fc4a5d6f5a4'),
  batch_id: 'B1165',
  mesin: 'M10',
  tanggal: 2026-06-09T18:02:00.000Z,
  shift: 2,
  inspektor: 'Eka',
  jumlah_diperiksa: 305,
  cacat_ditemukan: 6,
  jenis_cacat: [
    'warna'
  ]
}
```



Syntax:

```
db.target_kualitas.findOne()
```

Output:

```
< {  
  _id: ObjectId('69f84fe45c9f8fc4a5d6fb80'),  
  batch_id: 'B0001',  
  target_maks_cacat: 2  
}
```

## 5. Indeks

Syntax:

```
db.inspeksi.createIndex({ batch_id: 1 })  
db.inspeksi.createIndex({ mesin: 1, tanggal: 1 })  
db.target_kualitas.createIndex({ batch_id: 1 })
```

Output:

```
> db.inspeksi.createIndex({ batch_id: 1 })  
< batch_id_1  
> db.inspeksi.createIndex({ mesin: 1, tanggal: 1 })  
< mesin_1_tanggal_1  
> db.target_kualitas.createIndex({ batch_id: 1 })  
< batch_id_1
```

## BAB VI

### PIPELINE TUGAS MANDIRI

#### 6.1. Persentase Cacat per Batch

Tampilkan batch\_id, mesin, jumlah\_diperiksa, cacat\_ditemukan, dan persen\_cacat ( $\text{cacat\_ditemukan} / \text{jumlah\_diperiksa} * 100$ ). Urutkan berdasarkan persentase tertinggi.

Syntax:

```
db.inspeksi.aggregate([
  {
    $addFields: {
      persen_cacat: {
        $cond: [
          { $gt: ["$jumlah_diperiksa", 0] }, // Cek jika jumlah_diperiksa > 0
          { $multiply: [{ $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100] },
          0 // Jika nol, set persen_cacat jadi 0
        ]
      }
    }
  },
  {
    $sort: { persen_cacat: -1 }
  },
  {
    $limit: 10
  }
])
```

Output:

```

< {
  _id: ObjectId('69f8508ead73e55ddd25a0d2'),
  batch_id: 'B0929',
  mesin: 'M03',
  tanggal: 2026-05-05T11:52:00.000Z,
  shift: 2,
  inspektor: 'Andi',
  jumlah_diperiksa: 442,
  cacat_ditemukan: 44,
  jenis_cacat: [
    'gores',
    'pori'
  ],
  persen_cacat: 9.95475113122172
}
{
  _id: ObjectId('69f8508dad73e55ddd259eaf'),
  batch_id: 'B1656',
  mesin: 'M04',
  tanggal: 2026-05-06T01:08:00.000Z,
  shift: 1,
  inspektor: 'Andi',
  jumlah_diperiksa: 404,
  cacat_ditemukan: 40,
  jenis_cacat: [
    'dimensi',
    'retak',
    'warna'
  ],
  persen_cacat: 9.90998099809901
}
{
  _id: ObjectId('69f8508ead73e55ddd25a270'),
  batch_id: 'B0124',
  mesin: 'M05',
  tanggal: 2026-04-09T12:53:00.000Z,
  shift: 2,
  inspektor: 'Andi',
  jumlah_diperiksa: 182,
  cacat_ditemukan: 18,
  jenis_cacat: [
    'retak',
    'warna',
    'dimensi'
  ],

```

```

  persen_cacat: 9.89010989010989
}
{
  _id: ObjectId('69f8508ead73e55ddd25a214'),
  batch_id: 'B0865',
  mesin: 'M06',
  tanggal: 2026-05-18T09:05:00.000Z,
  shift: 2,
  inspektor: 'Dian',
  jumlah_diperiksa: 182,
  cacat_ditemukan: 18,
  jenis_cacat: [
    'pori',
    'warna'
  ],
  persen_cacat: 9.89010989010989
}
{
  _id: ObjectId('69f8508ead73e55ddd25a0cc'),
  batch_id: 'B0443',
  mesin: 'M01',
  tanggal: 2026-04-04T00:40:00.000Z,
  shift: 1,
  inspektor: 'Dian',
  jumlah_diperiksa: 427,
  cacat_ditemukan: 42,
  jenis_cacat: [
    'dimensi',
    'bengkok',
    'gores'
  ],
  persen_cacat: 9.836065573770492
}
{
  _id: ObjectId('69f8508ead73e55ddd25a3b1'),
  batch_id: 'B0797',
  mesin: 'M08',
  tanggal: 2026-05-15T03:22:00.000Z,
  shift: 1,
  inspektor: 'Citra',
  jumlah_diperiksa: 388,
  cacat_ditemukan: 38,
  jenis_cacat: [
    'pori',
    'bengkok',

```

```

      'gores'
    ],
    persen_cacat: 9.793814432989691
  }
  {
    _id: ObjectId('69f8508dad73e55ddd25a014'),
    batch_id: 'B1670',
    mesin: 'M88',
    tanggal: 2026-06-26T04:18:00.000Z,
    shift: 1,
    inspektor: 'Dian',
    jumlah_diperiksa: 429,
    cacat_ditemukan: 42,
    jenis_cacat: [
      'pori'
    ],
    persen_cacat: 9.79020979020979
  }
  {
    _id: ObjectId('69f8508dad73e55ddd25a2e3'),
    batch_id: 'B0121',
    mesin: 'M85',
    tanggal: 2026-05-06T09:33:00.000Z,
    shift: 2,
    inspektor: 'Eka',
    jumlah_diperiksa: 379,
    cacat_ditemukan: 37,
    jenis_cacat: [
      'retak'
    ],
    persen_cacat: 9.762532981530343
  }
}

```

```

}
{
  _id: ObjectId('69f8508dad73e55ddd25a09c'),
  batch_id: 'B0426',
  mesin: 'M85',
  tanggal: 2026-06-24T12:03:00.000Z,
  shift: 2,
  inspektor: 'Andi',
  jumlah_diperiksa: 287,
  cacat_ditemukan: 28,
  jenis_cacat: [
    'warna'
  ],
  persen_cacat: 9.75609756097561
}
{
  _id: ObjectId('69f8508dad73e55ddd259f02'),
  batch_id: 'B1935',
  mesin: 'M84',
  tanggal: 2026-05-13T07:45:00.000Z,
  shift: 2,
  inspektor: 'Citra',
  jumlah_diperiksa: 287,
  cacat_ditemukan: 28,
  jenis_cacat: [
    'gores',
    'bengkok',
    'dimensi'
  ],
  persen_cacat: 9.75609756097561
}
}

```

Stage \$addFields membuat fields baru bernama persen\_cacat, sedangkan stage %sort mengurutkan seluruh dokumen berdasarkan persen\_cacat dari persentase kerusakan paling parah. \$limit membatasi hasil keluaran agar hanya menampilkan 10 dokumen teratas.

## 6.2. Gabung dengan Target dan Tentukan Status Batch

Gabungkan koleksi inspeksi dengan target\_kualitas berdasarkan batch\_id, lalu hitung persentase cacat dan bandingkan dengan target\_maks\_cacat. Jika persen\_cacat > target\_maks\_cacat → status "NOT OK", sebaliknya "OK". Tampilkan juga selisihnya.

Syntax:

```

db.inspeksi.aggregate([
  {
    $lookup: {
      from: "target_kualitas",
      localField: "batch_id",
      foreignField: "batch_id",
      as: "target_info"
    }
  },
  {
    $unwind: {
      path: "$target_info",
      preserveNullAndEmptyArrays: true
    }
  }
])

```

```

    }
  },
  {
    $addFields: {
      persen_cacat: {
        $cond: {
          if: { $eq: ["$jumlah_diperiksa", 0] },
          then: 0,
          else: {
            $multiply: [
              { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
              100
            ]
          }
        }
      }
    },
    target_maks: "$target_info.target_maks_cacat"
  }
},
{
  $addFields: {
    status: {
      $cond: {
        if: { $gt: ["$persen_cacat", "$target_maks"] },
        then: "NOT OK",
        else: "OK"
      }
    }
  }
},
{
  $project: {
    _id: 0,
    batch_id: 1,
    mesin: 1,
    persen_cacat: 1,
    target_maks: 1,
    status: 1,
    selisih: {
      $subtract: [
        "$persen_cacat",
        { $ifNull: ["$target_maks", 0] }
      ]
    }
  }
},

```

```
{ $sort: { selisih: -1 } },
{ $limit: 10 }
])
```

Output:

```
< {
  batch_id: 'B0585',
  mesin: 'M09',
  persen_cacat: 9.880239520958083,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.880239520958083
}
{
  batch_id: 'B0129',
  mesin: 'M06',
  persen_cacat: 9.819639278557114,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.819639278557114
}
{
  batch_id: 'B1984',
  mesin: 'M04',
  persen_cacat: 9.512195121951219,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.512195121951219
}
{
  batch_id: 'B1703',
  mesin: 'M03',
  persen_cacat: 9.312638580931264,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.312638580931264
}
{
  batch_id: 'B1266',
  mesin: 'M03',
  persen_cacat: 9.289617486338798,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.289617486338798
}
{
  batch_id: 'B0607',
  mesin: 'M06',
  persen_cacat: 9.090909090909092,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.090909090909092
}
{
  batch_id: 'B0970',
  mesin: 'M10',
  persen_cacat: 8.875739644970414,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 7.875739644970414
}
{
  batch_id: 'B0861',
  mesin: 'M10',
  persen_cacat: 9.859154929577464,
  target_maks: 2,
  status: 'NOT OK',
  selisih: 7.859154929577464
}
{
  batch_id: 'B1450',
  mesin: 'M01',
  persen_cacat: 9.782608695652174,
  target_maks: 2,
  status: 'NOT OK',
  selisih: 7.782608695652174
}
{
  batch_id: 'B0732',
  mesin: 'M03',
  persen_cacat: 8.759124087591241,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 7.759124087591241
}
```

Stage \$lookup menghubungkan batch\_id yang ada pada koleksi target\_kualitas dan inspeksi menjadi target\_info, lalu stage \$unwind membongkar array tersebut menjadi objek tunggal. Stage \$addFields menghitung persen\_cacat. Stage \$addFields memberikan label “NOT OK”, jika persen\_cacat lebih besar dari target\_maks, jika tidak, maka akan “OK” diberikan. \$project merapikan output dengan hanya menampilkan kolom penting,

sekaligus menghitung selisih antara cacat dan target. Stage \$sort dan \$limit mengurutkan berdasarkan selisih tersebar ke terkecil dan hanya dibatasi 10 dokumen.

### 6.3. Jumlah Batch NOT OK per Mesin per Minggu

Ekstrak minggu dari tanggal menggunakan \$week, lalu filter hanya batch dengan status NOT OK, kemudian group per mesin dan minggu.

Syntax:

```
db.inspeksi.aggregate([
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"]
  }, 100 ] }
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] },
    then: "NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: {
      mesin: "$mesin",
      minggu: { $week: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    jumlah_NOT_OK: { $sum: 1 }
  }},
  { $sort: { "_id.tahun": 1, "_id.minggu": 1, "_id.mesin": 1 } }
])
```

Output:

```
< {
  _id: {
    mesin: 'M01',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M02',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M03',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
}
{
  _id: {
    mesin: 'M04',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M05',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
}

{
  _id: {
    mesin: 'M06',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M07',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
}
{
  _id: {
    mesin: 'M08',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 3
}
{
  _id: {
    mesin: 'M09',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 3
}
{
  _id: {
    mesin: 'M10',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
}
}
```

Stage %lookup menggabungkan koleksi inspeksi dengan target\_kualitas berdasarkan batch\_id, lalu \$unwind memecah array hasil gabungan menjadi objek tunggal. Stage \$addFields menghitung persentase cacat dan menentukan status “NOT OK” jika persentase cacat lebih besar dari pada target maksimum, %match menyaring data agar hanya menampilkan dokumen yang berstatus “NOT OK”. Stage \$group mengelompokkan data berdasarkan mesin dan waktu serta menghitung total “NOT OK”, lalu stage \$sort mengurutkan hasil berdasarkan waktu.



#### 6.4. Top 3 Mesin dengan Jumlah NOT OK Terbanyak (dalam satu bulan tertentu)

Syntax:

```
db.inspeksi.aggregate([
  { $match: {
    tanggal: { $gte: ISODate("2026-05-01"), $lt: ISODate("2026-06-01") }
  }},
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"]
  }, 100 ] }
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] },
    then: "NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: "$mesin",
    total_NOT_OK: { $sum: 1 }
  }},
  { $sort: { total_NOT_OK: -1 } },
  { $limit: 3 }
])
```

Output:

```
< {
  _id: 'M09',
  total_NOT_OK: 21
}
{
  _id: 'M10',
  total_NOT_OK: 20
}
{
  _id: 'M02',
  total_NOT_OK: 19
}
```

Stage \$match menyaring data hanya untuk periode bulan Mei 2026, stage \$lookup dan \$unwind sebagai integrasi data, lalu \$addFields menambahkan fields baru. Stage \$match selanjutnya menyaring data agar hanya data berstatus “NOT OK” yang ditampilkan, stage \$group menghitung total “NOT OK” untuk setiap mesin, dan diurutkan serta dibatasi oleh stage \$sort dan \$limit.

## 6.5. Ringkasan Bulanan dengan \$facet

Syntax:

```
db.inspeksi.aggregate([
  { $match: { tanggal: { $gte: ISODate("2026-06-01"), $lt: ISODate("2026-07-01") } } },
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  } },
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] },
    status: { $cond: { if: { $gt: [ { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] }, "$target_info.target_maks_cacat" ] }, then: "NOT OK", else: "OK" } } }
  } },
  { $facet: {
    "total_batch": [
      { $count: "count" }
    ],
    "rata_cacat": [
      { $group: { _id: null, avg_cacat: { $avg: "$persen_cacat" } } }
    ],
    "mesin_terburuk": [
      { $match: { status: "NOT OK" } },
      { $group: { _id: "$mesin", jumlah: { $sum: 1 } } },
      { $sort: { jumlah: -1 } },
      { $limit: 3 }
    ]
  } }
])
```

Output:

```
< {
  total_batch: [
    {
      count: 493
    }
  ],
  rata_cacat: [
    {
      _id: null,
      avg_cacat: 4.904385958864056
    }
  ],
  mesin_terburuk: [
    {
      _id: 'M09',
      jumlah: 21
    },
    {
      _id: 'M08',
      jumlah: 19
    },
    {
      _id: 'M01',
      jumlah: 14
    }
  ]
}
```

Stage \$facet menjalankan tiga analisis berbeda, yaitu menghitung jumlah keseluruhan batch yang tersisa pada bulan tersebut, lalu menghitung rata-rata persentase cacat, dan mengidentifikasi 3 mesin dengan frekuensi status “NOT OK” terbanyak.

## **BAB VII**

### **ANALISIS TUGAS MANDIRI**

Database tugas5\_225443025 digunakan untuk mengaudit kualitas produk batch demi batch dengan membandingkan hasil lapangan terhadap ambang batas toleransi kualitas. Data menunjukkan adanya beberapa kejadian di mana persentase cacat melampaui target maksimal, sehingga masuk pada status "NOT OK". Diperlukan perbaikan untuk menjaga standar kualitas perusahaan.

## BAB VIII

### DESAIN DATA LATIHAN MANDIRI

#### 8.1. Membuat Database

Syntax:

```
use latihan5_225443025
```

Output:

```
test> use latihan5_225443025
switched to db latihan5_225443025
```

#### 8.2. Membuat Dokumen

Syntax:

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

NIM = "225443025"
client = MongoClient('mongodb://localhost:27017')

db = client[f'latihan5_{225443025p}']

db.data_sensor.drop()

sensor_list = [f'S{i:03d}' for i in range(1, 21)] # S001 sampai S020
kategori_list = ["suhu", "tekanan", "kelembaban", "getaran", "arus"]

docs = []
start_date = datetime(2026, 4, 1, 0, 0)

for i in range(5000):
    kategori = random.choice(kategori_list)

    if kategori == "suhu":
        nilai = round(random.uniform(25.0, 95.0), 2)
    elif kategori == "kelembaban":
        nilai = round(random.uniform(40.0, 99.0), 2)
    else:
        nilai = round(random.uniform(10.0, 150.0), 2)

    timestamp = start_date + timedelta(minutes=random.randint(0, 60*24*30))

    doc = {
        "sensor_id": random.choice(sensor_list),
        "nilai": nilai,
```

```

        "timestamp": timestamp,
        "category": kategori
    }

    docs.append(doc)

    if len(docs) >= 500:
        db.data_sensor.insert_many(docs)
        docs.clear()

    if docs:
        db.data_sensor.insert_many(docs)

    print(f'Database: latihan5_{NIM}')
    print("Data sensor selesai dimasukkan. Total:", db.data_sensor.count_documents({}),
          "dokumen.")

```

Output:

```

PS C:\Users\Renisa123\Documents\bsd> & C:\Users\Renisa123\AppData\Local\Microsoft\Windows\Apps\python.exe C:\Users\Renisa123\Documents\bsd/p.py
Database: latihan5_225443025
Data sensor selesai dimasukkan. Total: 5000 dokumen.

```

### 8.3. Memverifikasi Data

Syntax:

```
db.data_sensor.countDocuments()
```

Output:

```

latihan5_225443025> db.data_sensor.countDocuments()
5000

```

## BAB IX

### PIPELINE LATIHAN MANDIRI

#### 9.1. Menampilkan Rata-Rata Nilai

Menampilkan rata-rata nilai per sensor\_id pada 7 hari terakhir.

Syntax:

```
db.data_sensor.aggregate([
  {
    $match: {
      timestamp: {
        $gte: new Date("2026-04-27T00:00:00Z")
      }
    }
  },
  {
    $group: {
      _id: "$sensor_id",
      rata_rata: { $avg: "$nilai" }
    }
  },
  {
    $sort: { rata_rata: 1 }
  }
])
```

Output:

```
< {
  _id: 'S005',
  rata_rata: 62.308928571428574
}
{
  _id: 'S014',
  rata_rata: 68.72916666666667
}
{
  _id: 'S011',
  rata_rata: 70.58324999999999
}
{
  _id: 'S008',
  rata_rata: 71.76769230769231
}
{
  _id: 'S009',
  rata_rata: 72.89117647058823
}
```

```
{
  _id: 'S010',
  rata_rata: 73.05133333333333
}
{
  _id: 'S017',
  rata_rata: 74.06911764705882
}
{
  _id: 'S016',
  rata_rata: 74.2897619047619
}
{
  _id: 'S012',
  rata_rata: 74.48027027027027
}
{
  _id: 'S013',
  rata_rata: 76.27324324324324
}
```

Stage \$match berfungsi untuk menyaring data sensor agar hanya mengambil dokumen yang memiliki waktu mulai tanggal 27 April 2026 hingga data terbaru (7 hari terakhir), pengelompokkan berdasarkan sensor\_id dan penghitungan nilai rata-rata dari kolom nilai dilakukan oleh stage \$group. Stage \$sort menyusun hasil berdasarkan nilai rata\_rata dari yang terkecil hingga terbesar.

## 9.2. Menampilkan Sensor Yang Memiliki Nilai Di Atas 90

Menampilkan sensor yang memiliki nilai di atas 90 minimal 3 kali (gunakan \$group dengan \$push atau \$sum kondisional).

Syntax:

```
db.data_sensor.aggregate([
  {
    $match: {
      nilai: { $gt: 90 }
    }
  },
  {
    $group: {
      _id: "$sensor_id",
      jumlah_data: { $sum: 1 }
    }
  },
  {
    $match: {
      jumlah_data: { $gte: 3 }
    }
  },
  {
    $sort: {
      jumlah_data: 1
    }
  }
])
```



Output:

```
< {
  _id: 'S018',
  jumlah_data: 62
}
{
  _id: 'S002',
  jumlah_data: 65
}
{
  _id: 'S008',
  jumlah_data: 66
}
{
  _id: 'S019',
  jumlah_data: 69
}
{
  _id: 'S007',
  jumlah_data: 71
}
{
  _id: 'S004',
  jumlah_data: 72
}
{
  _id: 'S006',
  jumlah_data: 74
}
{
  _id: 'S017',
  jumlah_data: 74
}
{
  _id: 'S003',
  jumlah_data: 74
}
{
  _id: 'S009',
  jumlah_data: 75
}
```

Stage \$match menyaring data agar hanya mengambil dokumen dengan nilai di atas 90, lalu \$group mengelompokkan data berdasarkan identitas sensor\_id dan menghitung total kemunculan data dengan nilai di atas 90 ke dalam jumlah\_data. Selanjutnya, \$match menyaring hasil pengelompokkan agar hanya menampilkan sensor yang memiliki jumlah data di atas 90 sebanyak 3 kali atau lebih. Stage \$sort menyusun daftar sensor berdasarkan jumlah\_data dimulai dari yang terkecil.

### 9.3. Menambahkan Field Jam

Menambahkan field jam dan menghitung nilai rata-rata per jam per sensor.

Syntax:

```
db.data_sensor.aggregate([  
  
  {  
    $addFields: {  
      jam: { $hour: "$timestamp" }  
    }  
  },  
  {  
    $group: {  
      _id: { sensor: "$sensor_id", jam: "$jam" },  
      rata_rata_jam: { $avg: "$nilai" }  
    }  
  },  
  { $sort: { "_id.sensor": 1, "_id.jam": 1 } }  
])
```

Output:

```
< {
  _id: {
    sensor: 'S001',
    jam: 0
  },
  rata_rata_jam: 77.09333333333333
}
{
  _id: {
    sensor: 'S001',
    jam: 1
  },
  rata_rata_jam: 83.25
}
{
  _id: {
    sensor: 'S001',
    jam: 2
  },
  rata_rata_jam: 71.14714285714285
}
{
  _id: {
    sensor: 'S001',
    jam: 3
  },
  rata_rata_jam: 84.0875
}
{
  _id: {
    sensor: 'S001',
    jam: 4
  },
  rata_rata_jam: 68.32692307692308
}
}

{
  _id: {
    sensor: 'S001',
    jam: 5
  },
  rata_rata_jam: 84.49166666666666
}
{
  _id: {
    sensor: 'S001',
    jam: 6
  },
  rata_rata_jam: 60.38636363636363
}
{
  _id: {
    sensor: 'S001',
    jam: 7
  },
  rata_rata_jam: 51.702
}
{
  _id: {
    sensor: 'S001',
    jam: 8
  },
  rata_rata_jam: 76.11125
}
{
  _id: {
    sensor: 'S001',
    jam: 9
  },
  rata_rata_jam: 70.0975
}
}
```

Stage \$addFields digunakan untuk membuat kolom baru bernama jam dengan mengambil atribut nilai jam dari atribut timestamp pada setiap dokumen, lalu \$group mengelompokkan data, dan menghitung nilai rata-rata jam tersebut. Stage \$sort menyusun hasil berdasarkan identitas sensor dan urutan jam dari yang terkecil hingga terbesar.

## 9.4. Membuat Koleksi Baru

Syntax:

```
var info = [];
for (var i = 1; i <= 20; i++) {
  var id = "S" + i.toString().padStart(3, '0');
  info.push({
    sensor_id: id,
    nama_sensor: "Sensor-" + id
  });
}
db.sensor_info.insertMany(info);

db.data_sensor.aggregate([
  {
    $group: {
      _id: "$sensor_id",
      rata_rata: { $avg: "$nilai" }
    }
  },
  {
    $lookup: {
      from: "sensor_info",
      localField: "_id",
      foreignField: "sensor_id",
      as: "detail_sensor"
    }
  },
  { $unwind: "$detail_sensor" },
  {
    $project: {
      _id: 1,
      rata_rata: 1,
      nama_sensor: "$detail_sensor.nama_sensor"
    }
  }
])
```

Output:

```
[
  {
    _id: 'S008',
    rata_rata: 72.52095454545454,
    nama_sensor: 'Sensor-S008'
  },
  {
    _id: 'S004',
    rata_rata: 72.49790441176471,
    nama_sensor: 'Sensor-S004'
  },
  {
    _id: 'S006',
    rata_rata: 72.71357976653697,
    nama_sensor: 'Sensor-S006'
  },
  {
    _id: 'S005',
    rata_rata: 71.92840304182509,
    nama_sensor: 'Sensor-S005'
  },
  {
    _id: 'S015',
    rata_rata: 74.7511673151751,
    nama_sensor: 'Sensor-S015'
  },
  },
  {
    _id: 'S020',
    rata_rata: 75.14873469387754,
    nama_sensor: 'Sensor-S020'
  },
  {
    _id: 'S002',
    rata_rata: 74.72474178403756,
    nama_sensor: 'Sensor-S002'
  },
  {
    _id: 'S012',
    rata_rata: 76.2110970464135,
    nama_sensor: 'Sensor-S012'
  },
  {
    _id: 'S003',
    rata_rata: 70.80725925925925,
    nama_sensor: 'Sensor-S003'
  },
  {
    _id: 'S009',
    rata_rata: 72.78728813559322,
    nama_sensor: 'Sensor-S009'
  },
  {
```

Membuat daftar nama 20 sensor dengan format “Sensor”- + id, lalu menyimpannya ke sensor\_info.

## **BAB X**

### **ANALISIS LATIHAN MANDIRI**

Database tugas5\_225443025 bertindak sebagai pusat pemantauan sensor industri yang menyimpan ribuan catatan pembacaan suhu, tekanan, arus, getaran, dan kelembaban. Temuan data mengidentifikasi adanya nilai rata-rata pada setiap perangkat yang terdaftar dalam sistem.

## **BAB XI**

### **KESIMPULAN**

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan Aggregation framework sangat efektif dalam mengubah data mentah menjadi informasi yang terstruktur sesuai kebutuhan. Melalui perintah yang tepat, ribuan dokumen dapat diringkas menjadi lebih padat dan mudah dipahami.

Selain itu, hasil pengolahan data agregasi memberikan gambaran objektif mengenai kinerja sistem secara menyeluruh, sehingga dapat digunakan sebagai dasar dalam mengidentifikasi perbedaan yang buruk, melakukan perbaikan teknik, serta mendukung proses pengambilan keputusan untuk meningkatkan kualitas operasional. Secara keseluruhan, praktikum ini membuat gambaran mengenai penggunaan database NoSQL di dunia industri.

## **LAMPIRAN**

<https://drive.google.com/drive/folders/1KRgssiV5ahhr6D8Dmp8IajjScJYeRhrh>